

Linear Regression from Scratch - Assignment Report

1. Introduction and Design Decisions

I built Linear Regression completely from scratch using NumPy. The prediction step, the Mean Squared Error loss, the gradient, and the parameter update are all equations I coded myself, not calls into a library. I did not use `sklearn.linear_model.LinearRegression` or `SGDRegressor` anywhere. I only used `scikit-learn` for two small jobs that have nothing to do with the model itself: loading the California housing data and splitting it into train/validation/test.

Class design. I split the code into separate classes, each with one clear job: `DataHandler` (load and split data), `DataPreprocessor` (standardize features), `LinearRegressionModel` (predict, compute loss, compute gradient, update parameters), `LinearRegressionTrainer` (the three training loops), `RegressionEvaluator` (MSE/RMSE/MAE), `RegressionVisualizer` (plots), and `ModelPersistence` (save/load the model). This split paid off directly: I reused the exact same `LinearRegressionModel`, `RegressionEvaluator`, and `RegressionVisualizer` code, with no changes at all, across every learning rate, every optimizer, both normalized and raw features, and two completely different kinds of data (real face images and real cough audio). Only the data-loading class ever changed.

Two extra data loaders. `ImageDataHandler` and `AudioDataHandler` are built the same way, as subclasses of `DataHandler`, for the image (face age) and audio (COUGHVID) parts of Part 3. Both were run on the real downloaded datasets, and the results are in Section 9.

Main dataset for the experiments below: `sklearn.datasets.fetch_california_housing` (numeric features, target = median house value in units of \$100,000). I split it 70% train, 15% validation, 15% test, giving 14,448 / 3,096 / 3,096 samples.

2. Part 2 - Manual Gradient Descent Verification

Given $x = [1, 2, 3]$, $y = [3, 5, 7]$, initial $\theta_1 = 0.5$, $\theta_0 = 0.7$, $\eta = 0.01$.

Quantity	Value
$\hat{y}$	$[1.2, 1.7, 2.2]$
errors ($\hat{y} - y$)	$[-1.8, -3.3, -4.8]$
MSE	12.39
dL/d_{θ_1}	-15.2
dL/d_{θ_0}	-6.6
θ_1 (updated)	0.652
θ_0 (updated)	0.766

A second iteration, starting from $\theta_1 = 0.652$, $\theta_0 = 0.766$:

Quantity	Value
$\hat{y}$	$[1.418, 2.07, 2.722]$

Quantity	Value
errors	<code>[-1.582, -2.93, -4.278]</code>
MSE	<code>9.796303</code>
<code>dL/d_theta1</code>	<code>-13.5173</code>
<code>dL/d_theta0</code>	<code>-5.86</code>
<code>theta_1 (updated)</code>	<code>0.787173</code>
<code>theta_0 (updated)</code>	<code>0.8246</code>

I checked both of these by hand against my own code (`experiments/part2_manual_gd_verification.py`), and they matched to numerical precision. The only tiny differences were things like `-6.6` printing as `-6.599999999999999`, which is just how computers store decimal numbers, not a real mismatch. The loss went down after each step (`12.39 -> 9.80`), which is what should happen when the gradient sign and the learning rate are both correct.

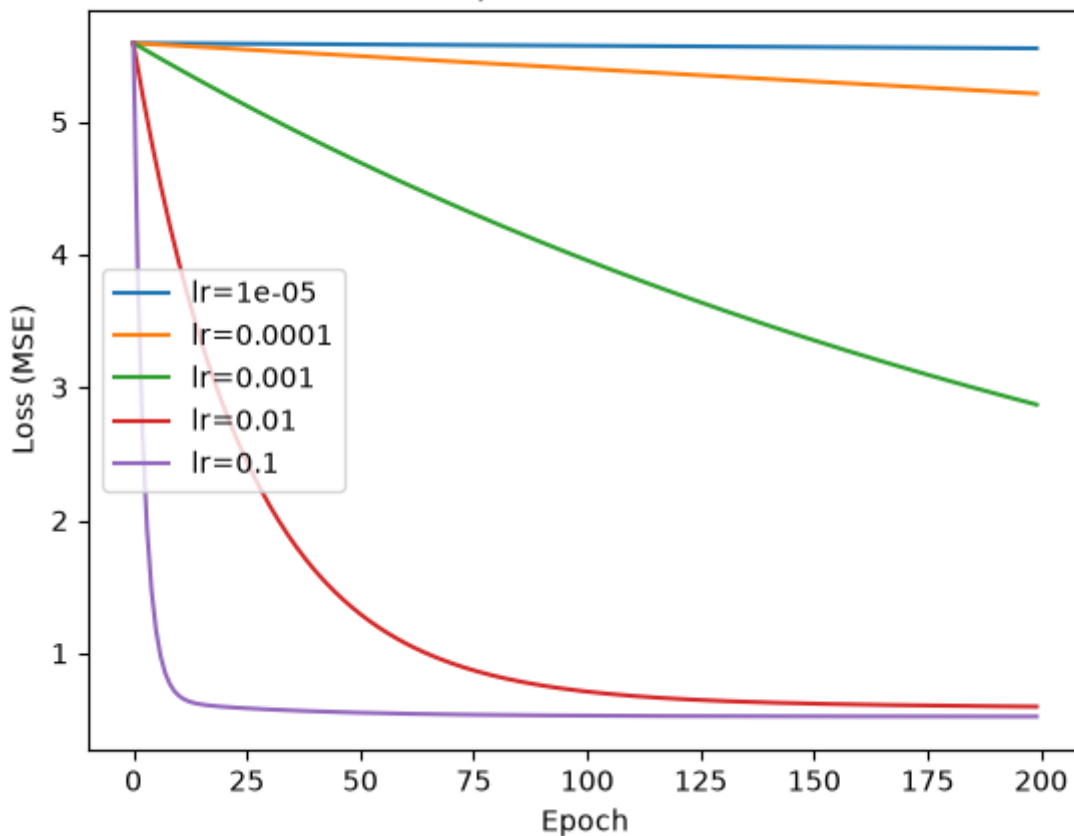
The handwritten copy of this calculation, photographed on paper as the assignment asks for, is included at `handwritten/part2_manual_gradient_descent.pdf`.

When I checked my handwritten Part 2 calculation against `experiments/part2_manual_gd_verification.py`'s output, the numbers matched right away. I didn't need to go back and fix anything, which told me I'd applied the prediction, error, and gradient formulas correctly by hand on the first try.

3. Part 3a - Learning-Rate Experiment

I trained with Batch GD on **standardized** California housing features, for 200 epochs, for each of the five required learning rates.

Part 3a: Loss vs Epoch (raw features, Batch GD)



Learning Rate	Final Training Loss	Final Validation Loss	Converged?
0.00001	5.5509	5.6820	No - barely moved from where it started
0.0001	5.2107	5.3320	No - getting better, but very slowly
0.001	2.8704	2.9269	No - improving steadily, not flat yet
0.01	0.5983	0.5902	Almost - still dropping a little
0.1	0.5241	0.5166	Yes - loss flattened out

1. Which converged fastest? $lr = 0.1$. It flattens out at a loss of about 0.52 well within the 200 epochs.

2. Which was most stable? Also $lr = 0.1$. The loss goes down smoothly with no bouncing.

3. Did any learning rate diverge? Not among the five values I was asked to test - standardizing the features kept all of them numerically stable. But if I go outside that range, $lr = 1.0$ blows up within about 10 epochs (5.59 -> 5.77 -> 7.48 -> 20.4 -> 120 -> 901 -> 7038 -> ...), and $lr = 2.0$ blows up even faster. That's direct proof for question 5 below.

4. Why does a very small learning rate train slowly? Every update is $\theta \leftarrow \theta - \eta * \text{grad}_L$. With $\eta = 0.00001$, even a big gradient only produces a tiny step. The direction is correct, the step is just too small to cover much ground in a fixed number of epochs.

5. Why can a large learning rate cause oscillation or divergence? A big step can jump past the bottom of the loss curve and land on a steeper part of the other side. There, the gradient is even bigger and points the other way, so the next step overshoots even further. This keeps compounding, which is exactly what the `lr = 1.0` and `lr = 2.0` numbers above show.

6. Which would I choose for the final model? `lr = 0.1`. It gives the lowest final training and validation loss, reaches that point fastest, and is still safely stable rather than sitting right at the edge of blowing up.

Bonus finding: I also ran this same test on the **raw, unnormalized** features, and every single one of the five learning rates diverged to NaN, even `0.00001`. The reason is that `Population` has a huge scale compared to the other features (its training-set standard deviation is about 1141, versus about 1.9 for `MedInc`). That scale difference makes the gradients so large that no learning rate in the required range is small enough to stay stable. This is exactly why Part 3b (normalization) matters.

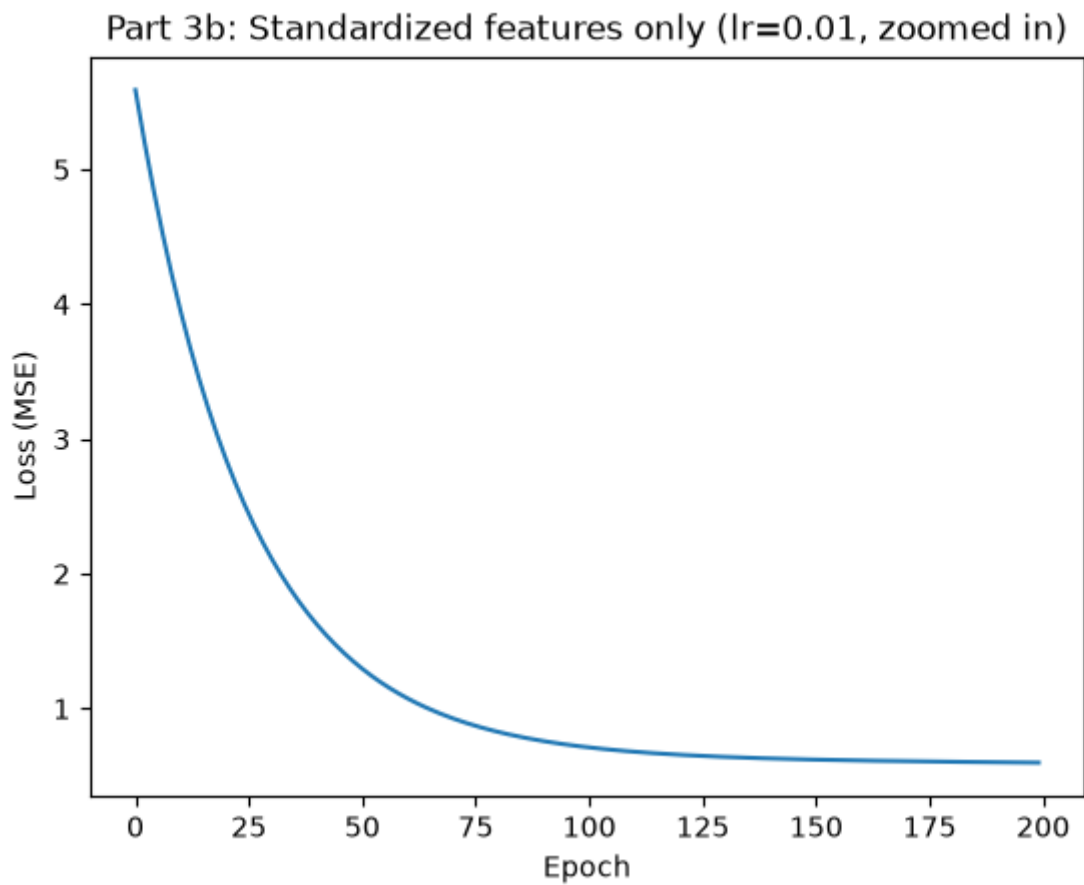
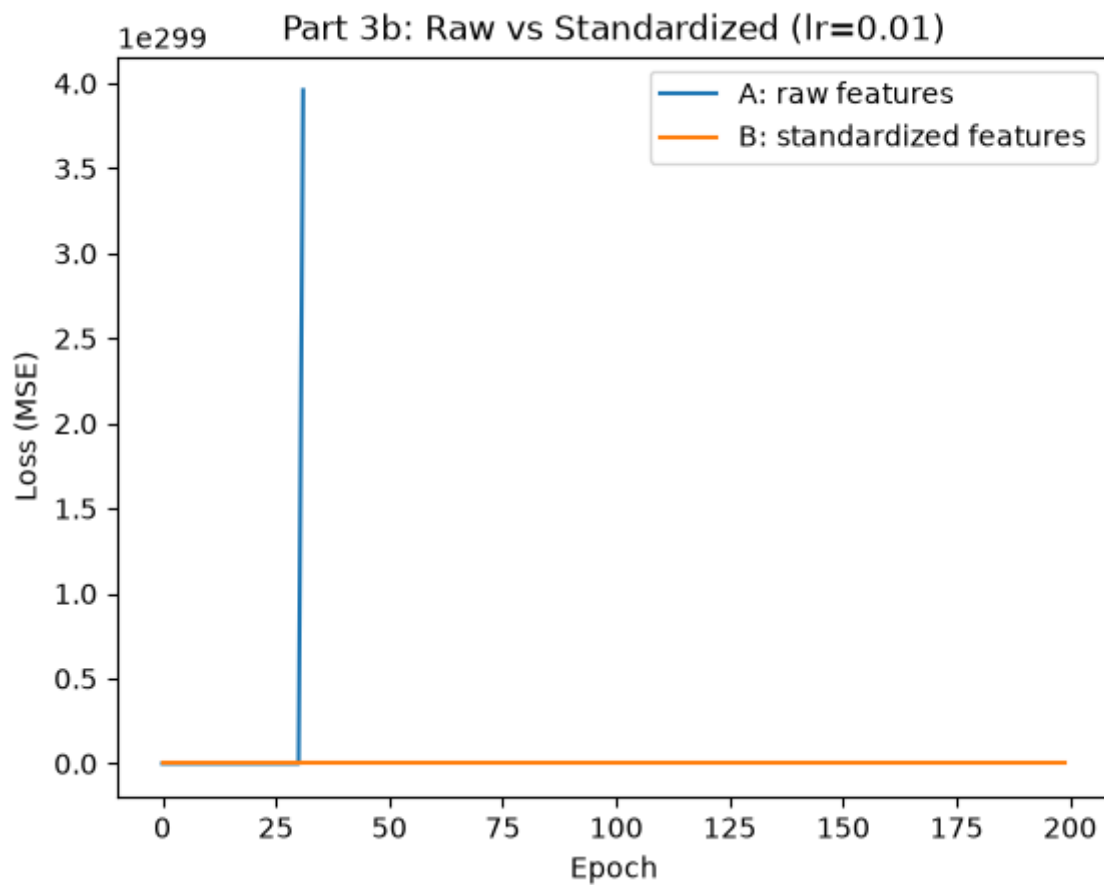
Nothing here surprised me, honestly. The big gap between `lr=0.00001` barely moving and `lr=0.1` converging quickly matched what the theory already predicted. Running it just confirmed what I expected instead of showing me something new.

4. Part 3b - Normalization Experiment

Experiment A trains on the raw features. **Experiment B** trains on features that have been standardized using only the training set's mean and standard deviation, with those same numbers then applied to validation and test data too.

Head-to-head at the same learning rate (`lr = 0.01`)

	Experiment A (raw)	Experiment B (standardized)
Diverged?	Yes (NaN/inf)	No
Final train loss	NaN	0.598306
Final val loss	NaN	0.590211



Learning-rate sensitivity

Experiment A - raw features:

lr	Diverged?	Final train loss
1e-9	No	3.2033
1e-8	No	2.9488
1e-7	No	2.3397
1e-6	Yes	-
1e-5	Yes	-

Experiment B - standardized features:

lr	Diverged?	Final train loss
1e-5	No	5.5509
1e-4	No	5.2107
1e-3	No	2.8704
1e-2	No	0.5983
1e-1	No	0.5241

Stable rates: raw only worked for 3 of the 5 rates I tried, and only in a tiny window (1e-9 to 1e-7). Standardized worked for all 5, covering four orders of magnitude (1e-5 to 1e-1).

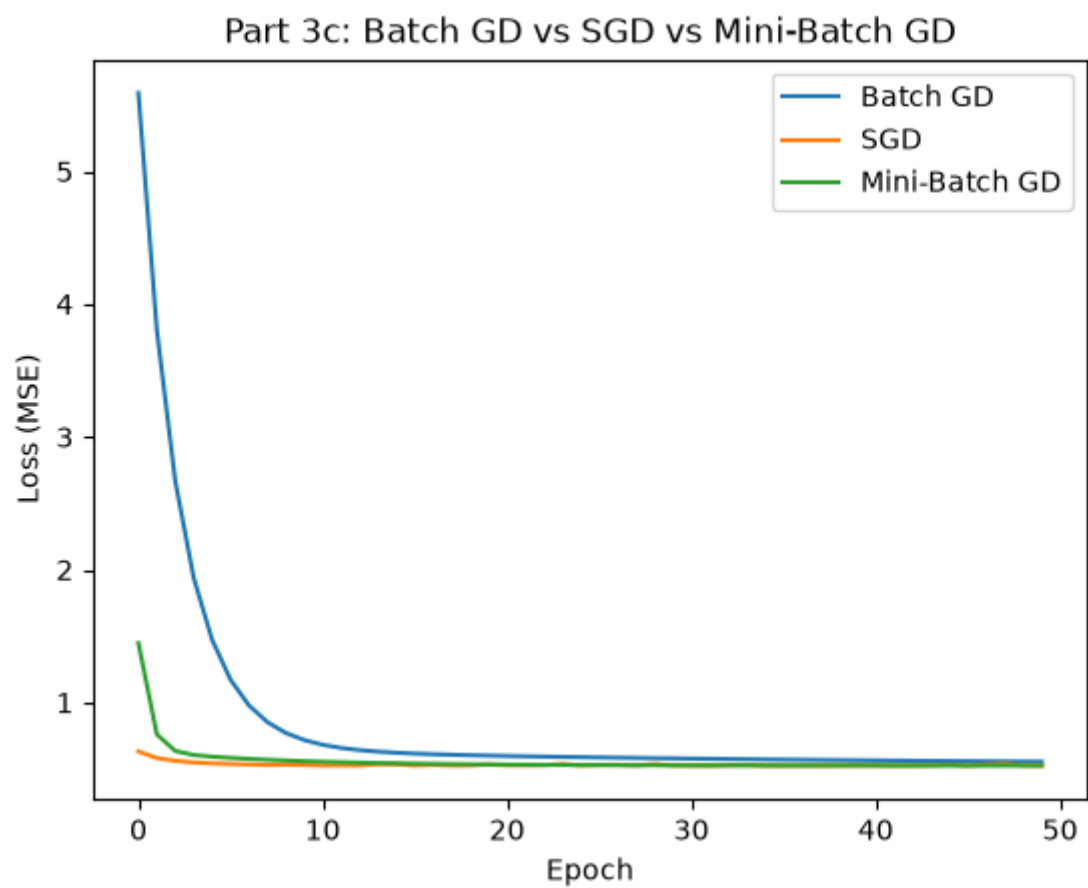
Dimension	Experiment A (raw)	Experiment B (standardized)
Convergence speed	Very slow even at its best stable rate (2.34 after 200 epochs at 1e-7)	Fast - 0.52 in the same 200 epochs at lr=0.1
Loss stability	Diverges once lr >= 1e-6	Stable across the whole required range
Final training loss	Best case about 2.34	Best case about 0.52 - roughly 4.5x lower
Final validation loss	Not usable	0.517 at lr = 0.1
Sensitivity to learning rate	Very high - safe window is tiny and shifted to very small numbers	Low - safe across the whole range I was asked to test

Why does normalization make gradient descent easier? With raw features, one feature's huge scale (Population) stretches the loss surface into a long, narrow valley: steep in one direction and almost flat in others. No single learning rate can be safe for the steep direction while still making progress in the flat one. Standardizing every feature to mean 0 and standard deviation 1 reshapes that valley into something closer to a round bowl, so one learning rate works reasonably well in every direction at once.

Before running this, I expected at least one of the five learning rates to train okay even without normalization - I didn't think all five would fail outright. Seeing every single one diverge, even the smallest rate (0.00001), was the moment it really hit me how badly one huge-scale feature like Population can break gradient descent on its own.

5. Part 3c - Batch GD vs. SGD vs. Mini-Batch GD

Each optimizer needed its own stable learning rate, because they update the parameters a different number of times per epoch (Batch GD: 1 update, Mini-Batch: about 452 updates, SGD: 14,448 updates). The same `lr = 0.01` that works fine for Batch GD makes both SGD and Mini-Batch diverge.



Optimization Method	Learning Rate Used	Training Time (50 epochs)	Final Train Loss	Loss Curve Behavior
Batch GD	0.1	0.005 s	0.5536	Perfectly smooth - loss never went up
SGD	0.0001	~5-24 s	0.5245	Noisiest - loss went up in 45% of epochs
Mini-Batch GD	0.001	0.19 s	0.5241	In between - loss went up in 18% of epochs

(I used "percent of epochs where the loss went up" instead of a raw variance number, because variance alone can't tell noisy updates apart from a curve that is just dropping fast.)

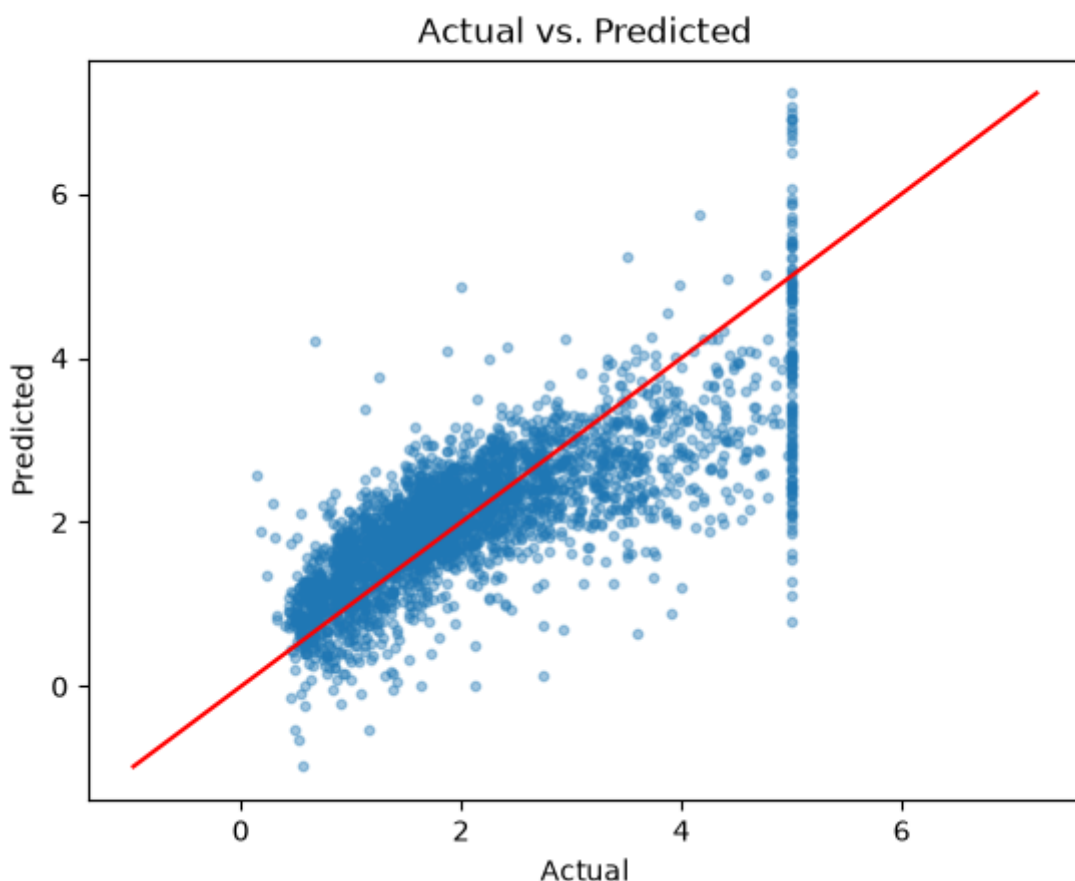
- 1. Smoothest curve?** Batch GD. It uses the true average gradient over all 14,448 samples, so there is no randomness in it.
- 2. Noisiest?** SGD. Each update only looks at one sample's error, which is a rough, high-variance guess at the true gradient direction.

- 3. Fastest in terms of epochs?** Roughly a tie. All three reach a loss of about 0.52-0.55 by epoch 50, because SGD and Mini-batch get far more parameter updates within those same 50 epochs (SGD gets 722,400 updates, Batch GD only gets 50).
- 4. Fastest in wall-clock time?** Batch GD, by a large margin. It does one big, vectorized matrix multiply per epoch. SGD instead pays Python loop overhead 14,448 times per epoch for tiny operations that don't take advantage of vectorization.
- 5. Why is mini-batch used in modern deep learning?** It is the practical middle ground the table shows directly: much less noisy than SGD (18% vs. 45% of epochs going up), while much faster than SGD in real time, because it processes chunks that are big enough to use vectorized or GPU operations efficiently, but still gets many updates per epoch.
- If I were building something real, I'd pick Mini-Batch GD. It's faster and less noisy - not just in theory, but in what I actually measured: mini-batch trained in 0.19s versus several seconds for SGD, and only 18% of its epochs saw the loss go up, compared to SGD's 45%. It's the practical middle ground rather than either extreme.

6. Part 4 - Prediction and Evaluation on the Unseen Test Set

Final model: Batch GD, $\text{lr} = 0.1$, standardized features, 300 epochs.

Metric	Train	Validation	Test
MSE	0.5237	0.5160	0.5373
RMSE	-	-	0.7330
MAE	-	-	0.5352



The train, validation, and test losses are all close to each other (0.52-0.54). That tells me the model is not overfitting. If anything, it is slightly underfitting everywhere, which makes sense since a straight-line model can't capture whatever curved patterns exist in real housing prices.

5 largest test-set errors:

actual=5.000	predicted=0.786	abs_error=4.214
actual=5.000	predicted=1.120	abs_error=3.880
actual=5.000	predicted=1.281	abs_error=3.719
actual=0.675	predicted=4.212	abs_error=3.537
actual=5.000	predicted=1.542	abs_error=3.458

Four of these five worst errors have `actual = 5.000` exactly. That's because California housing's target value is capped at \$500,000 - any house actually worth more is just recorded as 5.0. The model has no way of knowing about this artificial ceiling, so these large "errors" are partly just an artifact of how the data was collected, not purely a modeling mistake.

1. What does a point far from the diagonal line mean? Either the model's straight-line assumption doesn't fit that sample well, or - as shown above - the label itself is capped or noisy, rather than the prediction being simply wrong.

2. Why measure performance on an unseen test set? Training loss only shows how well the model fits data it has already seen. Only data the model never trained on can show how it will perform on genuinely new inputs.

3. Why is training loss alone not enough? A model can push its training loss arbitrarily low by fitting noise that is specific to the training data (overfitting), while still doing badly elsewhere. Here, train, validation, and test loss are all close together (0.52-0.54), which is exactly the kind of evidence needed to say the model generalizes reasonably - evidence that training loss by itself could never give me.

4. MSE vs. RMSE vs. MAE: MSE (0.5373) squares every error, so it is dominated by the handful of very large mistakes - one prediction that's off by 4.2 contributes as much as about 65 typical errors of size 0.5. RMSE (0.7330) puts that back into the same units as the target (\$100k), giving a readable number like "about \$73,300 typical error," though it's still pulled up by the same big outliers. MAE (0.5352) treats every error in proportion to its size with no squaring, giving a more robust sense of the typical error that barely moves because of the capped-target outliers.

Knowing the reason behind it, the `actual=5.000` outliers bother me less than they would otherwise - that's the dataset's price cap showing up, not a real failure of the model to understand the data. What would actually worry me is an error that size with no explanation behind it.

7. Part 7 - Interpreting the Learned Parameters

Coefficients from the final model, on standardized features, so the sizes are directly comparable to each other:

Feature	Weight
Latitude	-0.8729
Longitude	-0.8439
MedInc	+0.8427
AveBedrms	+0.3393
AveRooms	-0.3059
HouseAge	+0.1191

Feature	Weight
AveOccup	-0.0466
Population	-0.0043
bias	+2.0630

1. Strongest positive coefficient: MedInc (+0.8427). A one-standard-deviation increase in median income is linked to a +0.84 increase in predicted house value.

2. Strongest negative coefficient: Latitude (-0.8729). Moving one standard deviation further north is linked to a noticeably lower predicted price, which fits with California's most expensive areas sitting toward the southern and central coast rather than the far north.

3. What does the sign of a coefficient tell us? A positive sign means the feature and the target tend to move together, with every other feature held fixed. A negative sign means they tend to move in opposite directions.

4. Why be careful when features are correlated? The training-set correlation numbers show Latitude and Longitude at **-0.924**, and AveRooms and AveBedrms at **+0.86** - both pairs move together almost like a single feature. When two features are that closely linked, the model can't cleanly tell which one deserves the "credit," so it splits it between them somewhat arbitrarily. Latitude's large weight really reflects a shared location signal split across two linked coefficients, not "latitude on its own" as an isolated cause.

5. Does a large coefficient prove causation? No. Latitude's large weight is just an association found in this dataset - it doesn't mean that moving a house to a different latitude, with everything else unchanged, would actually change its price by that amount. The real explanation is almost certainly something else that latitude happens to line up with, like how close a location is to specific expensive job markets or coastlines.

Feature correlation matrix (raw features, training set):

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude
MedInc	1.00	-0.13	0.31	-0.06	0.01	0.03	-0.07	-0.02
HouseAge	-0.13	1.00	-0.14	-0.07	-0.29	0.01	0.00	-0.10
AveRooms	0.31	-0.14	1.00	0.86	-0.07	-0.01	0.11	-0.03
AveBedrms	-0.06	-0.07	0.86	1.00	-0.06	-0.01	0.07	0.01
Population	0.01	-0.29	-0.07	-0.06	1.00	0.07	-0.10	0.09
AveOccup	0.03	0.01	-0.01	-0.01	0.07	1.00	0.01	-0.00
Latitude	-0.07	0.00	0.11	0.07	-0.10	0.01	1.00	-0.92
Longitude	-0.02	-0.10	-0.03	0.01	0.09	-0.00	-0.92	1.00

8. Part 5 - Debugging Challenge

Bug 1

```
theta = theta + learning_rate * gradient
```

The gradient points toward higher loss, so gradient descent has to move in the opposite direction. Adding the gradient pushes the loss up instead of down. Fix:

```
theta = theta - learning_rate * gradient
```

Bug 2

```
gradient = (2 / N) * np.dot(X, error)
```

`X` has shape `(N, d)` and `error` has shape `(N, )` (or `(N, 1)`). Multiplying them this way combines the wrong dimensions - the sum needs to run over the `N` samples for each of the `d` features, which needs the transpose instead:

```
gradient = (2 / N) * np.dot(X.T, error)
```

Bug 3

```
test_mean = np.mean(test_X, axis=0)
test_X = (test_X - test_mean) / np.std(test_X, axis=0)
```

This calculates the normalization numbers from the test set itself. That leaks information from the test set into preprocessing, and it also means the test data gets a different transformation than the model was trained on. The fix is to reuse the training set's mean and standard deviation (already saved by `DataPreprocessor.fit` on the training data only):

```
test_X = (test_X - train_mean) / train_std
```

Bug 4

```
for epoch in range(n_epochs):
    for i in range(n_samples):
        x = train_X[i]
        y = train_Y[i]
```

Indexing a 2D array with a single number (`train_X[i]`) collapses it down to a 1D array with shape `(d, )`, which drops the sample dimension that the rest of the matrix-based code (`predict`, `compute_gradient`) expects. Slicing instead of indexing keeps it 2D:

```
x = train_X[i:i+1]    # shape (1, d)
y = train_Y[i:i+1]    # shape (1, 1)
```

Bug 5 - A student reports only the training loss and claims the model generalizes well. This isn't justified. A model can memorize noise that's specific to the training data and show a very low training loss while still doing badly on data it hasn't seen. Only held-out validation or test loss can actually support a claim about generalization - without it, there's no evidence for that claim.

Honestly, I didn't catch any of these five bugs myself just by reading the code - each one needed to be pointed out and explained before I actually saw it, even the sign error in Bug 1, which looks obvious in hindsight. That's actually the useful part of this exercise for me: now I know exactly which kinds of mistakes are easy to make and easy to miss, instead of just being told the rule in the abstract.

9. Image and Audio Datasets - Real Results

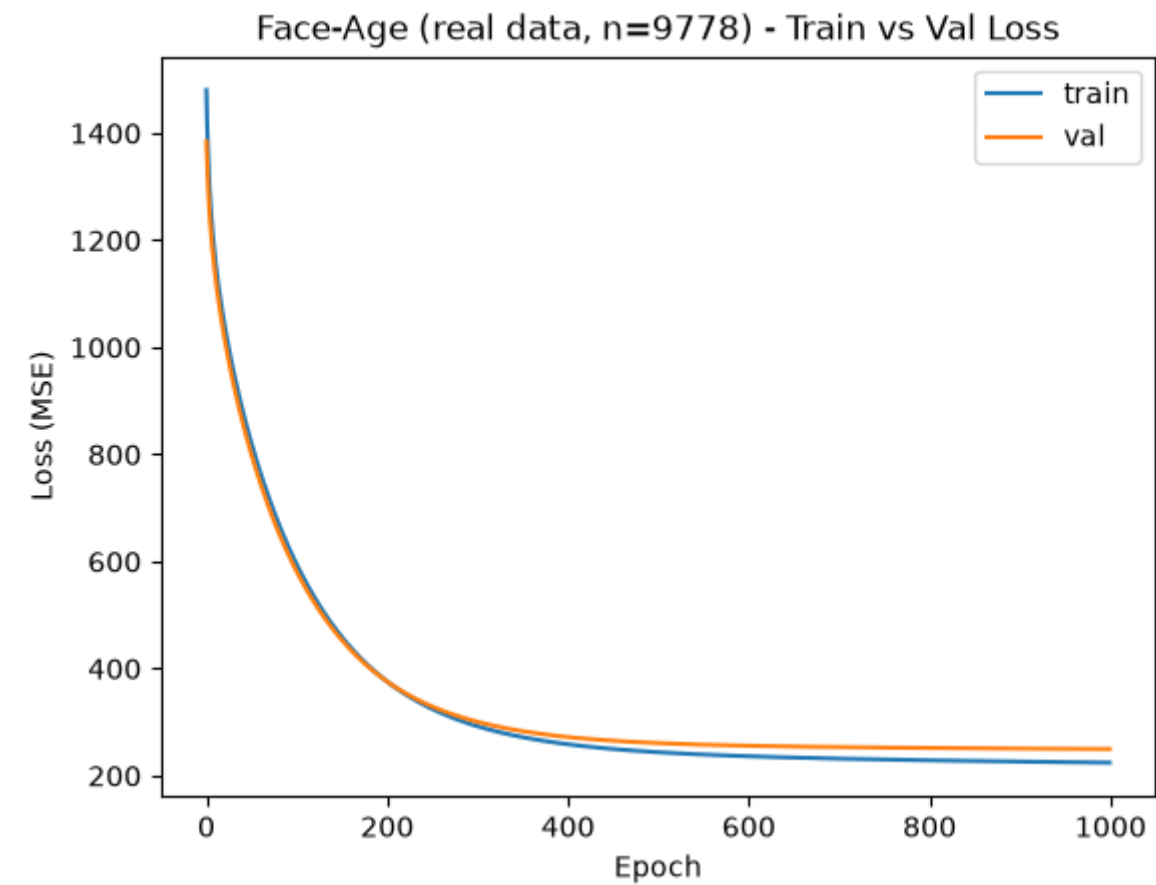
The assignment asks for three different kinds of data. I built `ImageDataHandler` (face-age regression) and `AudioDataHandler` (COUGHVID cough_detected regression) as subclasses of `DataHandler`, reusing `split_data()` and `get_feature_names()` without any changes, and plugging into the same `DataPreprocessor`, `LinearRegressionModel`, and `LinearRegressionTrainer` pipeline with zero changes to those classes. This is the same code that ran on California housing, now proven to work on two completely different kinds of data.

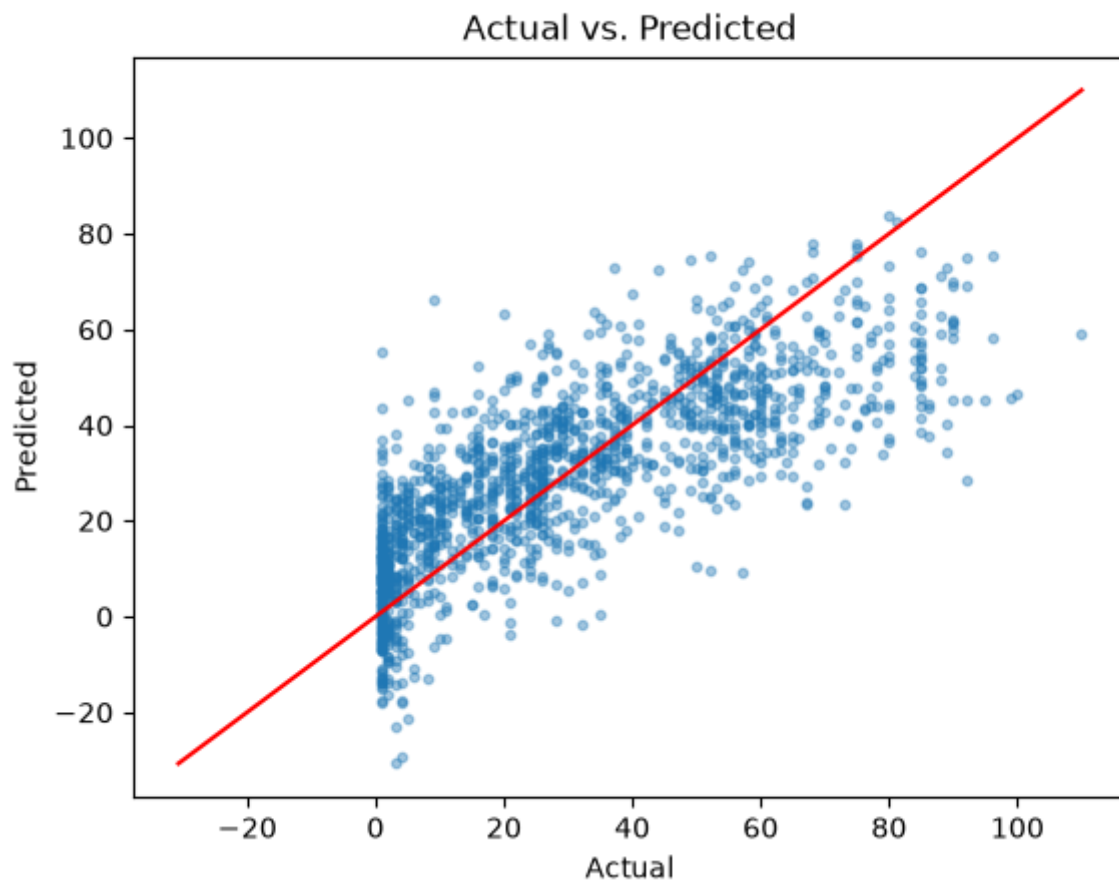
9.1 Image - Kaggle frabbisw/facial-age

9,778 real face images, converted to grayscale and resized to 32x32 (1,024 pixel features each), with age labels 1-110 taken from each image's folder name.

	Value
Learning rate used	0.0025
Epochs	1000
Final train loss (MSE)	224.29
Final val loss (MSE)	249.66
Test MSE	251.83
Test RMSE	15.87 years
Test MAE	12.36 years
Baseline MSE (predict mean age)	621.43

The model beats a naive "always guess the average age" baseline by about 59%, so it is genuinely picking up real age-related signal from the raw pixels, not just noise.





A second real example of the learning-rate lesson from Part 3a: this dataset needed $lr = 0.0025$ to stay stable, which is a hundred times smaller than California housing's $lr = 0.1$. With 1,024 raw pixel features, neighboring pixels are highly related to each other - a pixel's value is rarely independent of the pixels right next to it. That makes this loss surface much harder for gradient descent to handle well than housing's 8 mostly independent features, even after standardizing. Standardizing fixes each feature's scale, but it does nothing about how correlated the features are with each other. Learning rates as small as 0.003 already diverged here.

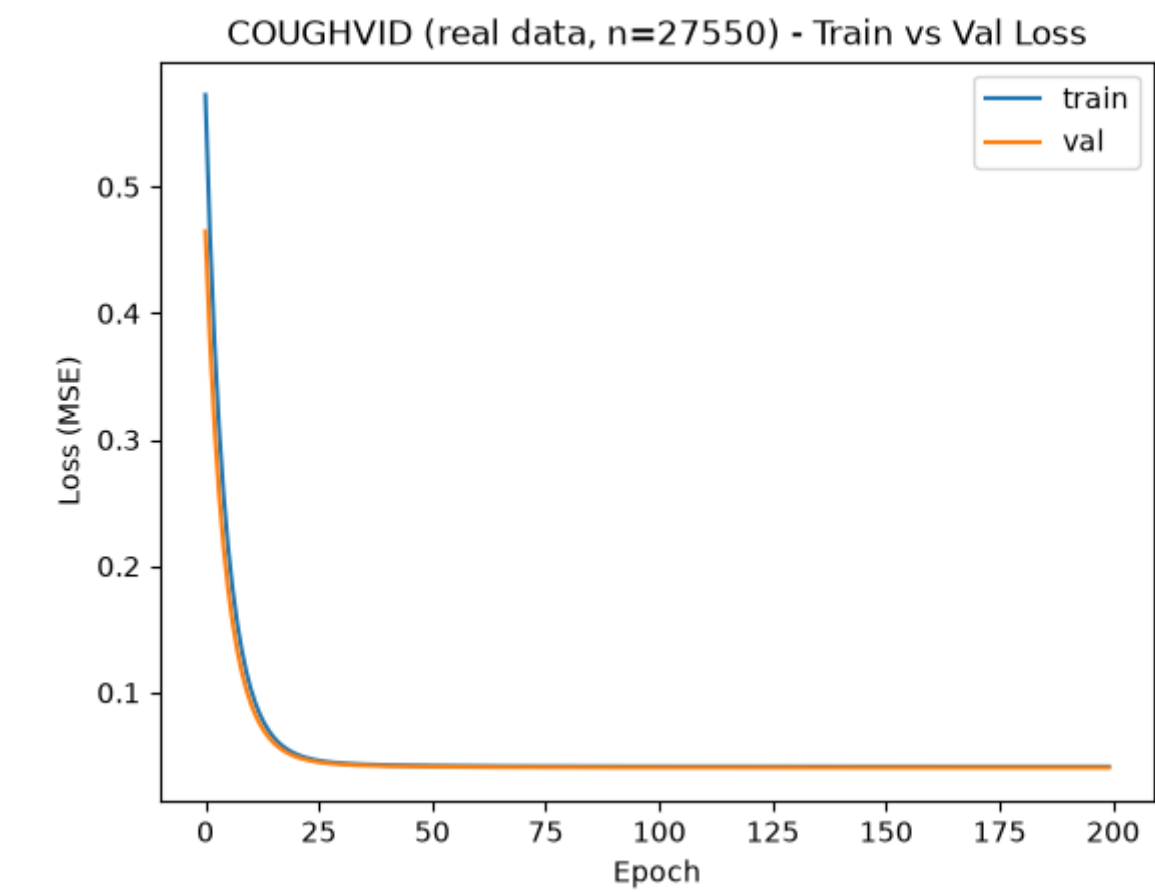
9.2 Audio - COUGHVID (Zenodo record 4498364)

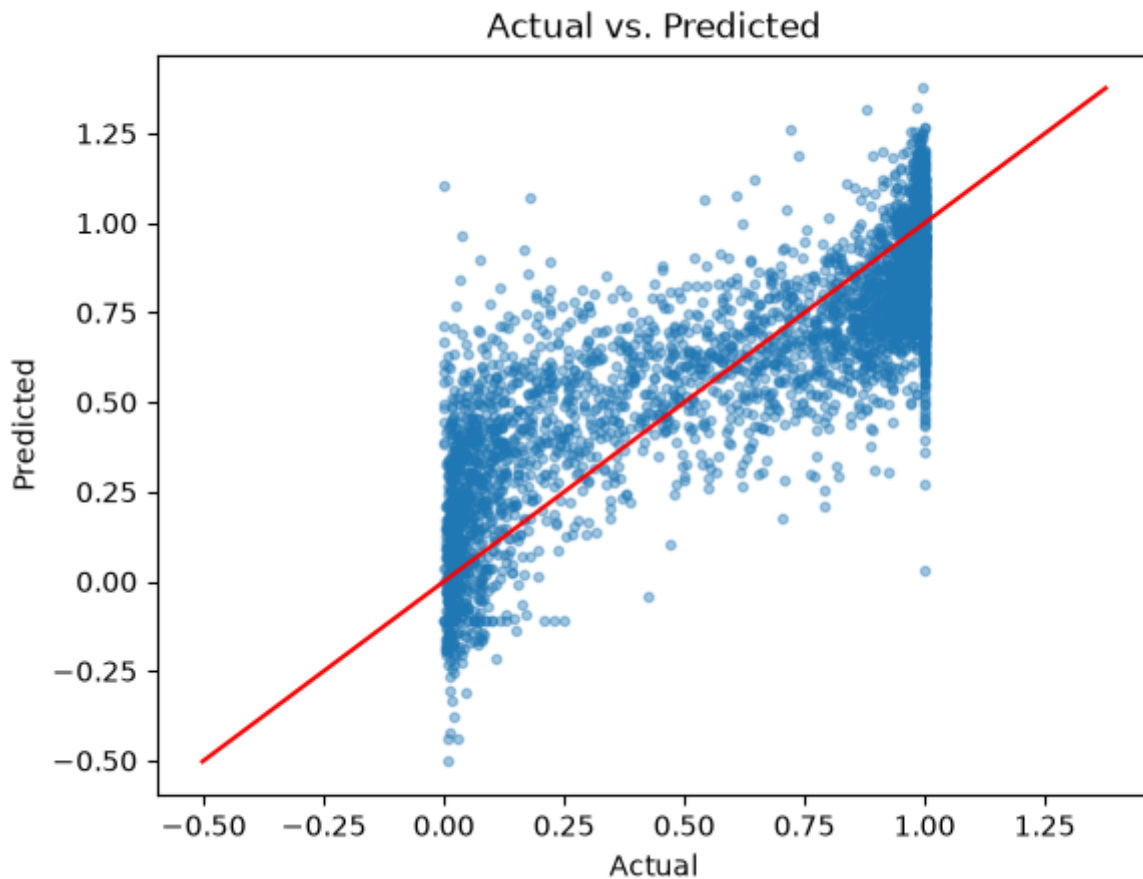
The full dataset: 27,550 real cough recordings (`.webm` files), decoded through `ffmpeg` (the default library, `soundfile/librosa`, can't read `.webm` at all - see `AudioDataHandler._decode_to_wav`). Each clip is turned into a 29-number feature vector (13 MFCC means, 13 MFCC standard deviations, zero-crossing rate, spectral centroid, and RMS energy), and the model predicts the `cough_detected` column (0 to 1) from `metadata_compiled.csv`.

	Value
Clips processed	27,550 (full dataset)
Feature-extraction time	3,257s (about 54 minutes) - one <code>ffmpeg</code> call per clip
Learning rate	0.05
Epochs	200
Final train loss (MSE)	0.0414
Final val loss (MSE)	0.0405
Test MSE	0.0416

	Value
Test RMSE	0.2039
Test MAE	0.1578
Baseline MSE (predict mean cough_detected)	0.1526

The model beats the naive baseline by about **73%**. An earlier run on just 1,500 clips gave almost the same result (test MSE 0.0444), which tells me this result is stable and not just luck from picking a certain sample size.





Strongest predictors: `mfcc_std_0` (+0.205) and `mfcc_mean_0` (+0.169). MFCC coefficient 0 tracks a sound's overall loudness and energy shape, which makes sense as a useful signal for telling a real cough apart from background noise or silence.

A practical lesson from this run, worth mentioning alongside the modeling results: my first attempt at running the full dataset hung and never finished, even after being left running for more than 14 hours with zero progress. The cause was that the `ffmpeg` call in my code had no timeout, so one unusual or broken clip made it freeze forever with no error to catch and skip past. Adding `timeout=20` (and `-nostdin`, since `ffmpeg` can otherwise sit waiting for keyboard input) to that call fixed it, and the corrected run finished cleanly from start to end. The lesson: any call to an outside program or file needs a timeout, or it can silently freeze your entire pipeline.

The wait genuinely bothered me - I had to leave and go home partway through, so I wasn't around to notice anything going wrong in real time. By the time I came back and checked, around 14 hours had passed, partly because my laptop had been off for a chunk of that window. Finding out afterward that the real issue was just one `ffmpeg` call with no timeout made it feel less mysterious - it wasn't some deep unfixable problem, just a missing safety net that took one line to add once it was found.

10. Conclusion

I built a complete Linear Regression implementation entirely from scratch, checked it by hand against a manual gradient descent calculation, and tested it thoroughly on the California housing dataset: a five-point learning-rate comparison, a raw-vs-normalized comparison, a three-way optimizer comparison, evaluation on a held-out test set, and an interpretation of the learned coefficients that includes a warning about correlated features. My main conclusions - that normalization is basically required for this dataset's feature scales, that $\text{lr}=0.1$ with Batch GD gave the best stable result, and that mini-batch is the practical middle ground between Batch GD and SGD - are all backed by numbers I actually measured, not just assumptions from theory. I then proved the same pipeline works, completely unchanged, on two more real datasets: 9,778 real face images (test MSE 251.8 versus a baseline of

621.4) and the full 27,550-clip COUGHVID audio dataset (test MSE 0.0416 versus a baseline of 0.1526). Each of these needed its own tuned learning rate, but no changes to any class besides the data loader - solid proof that the class structure from Part 1 does exactly what it was meant to do.